

Programación de microcontroladores

Salidas analógicas: DAC y PWM

Juan José Londoño, M.Sc.

Introducción

En muchos sistemas embebidos no solo se necesita medir señales del mundo físico, sino también generarlas o controlarlas.

En muchas aplicaciones el microcontrolador debe interactuar activamente con el entorno, produciendo señales que controlen dispositivos físicos. Esto implica generar señales que se comporten como **voltajes analógicos**, aun cuando el microcontrolador internamente trabaja de manera digital.

Introducción

Generar una señal analógica desde un sistema digital permite, por ejemplo:

- Controlar brillo de un LED.
- Controlar velocidad de un motor.
- Generar audio.
- Generar una señal de control o referencia analógica.

En todos estos casos, el microcontrolador debe transformar un valor digital interno en una señal analógica que pueda interactuar con el mundo físico.

Introducción

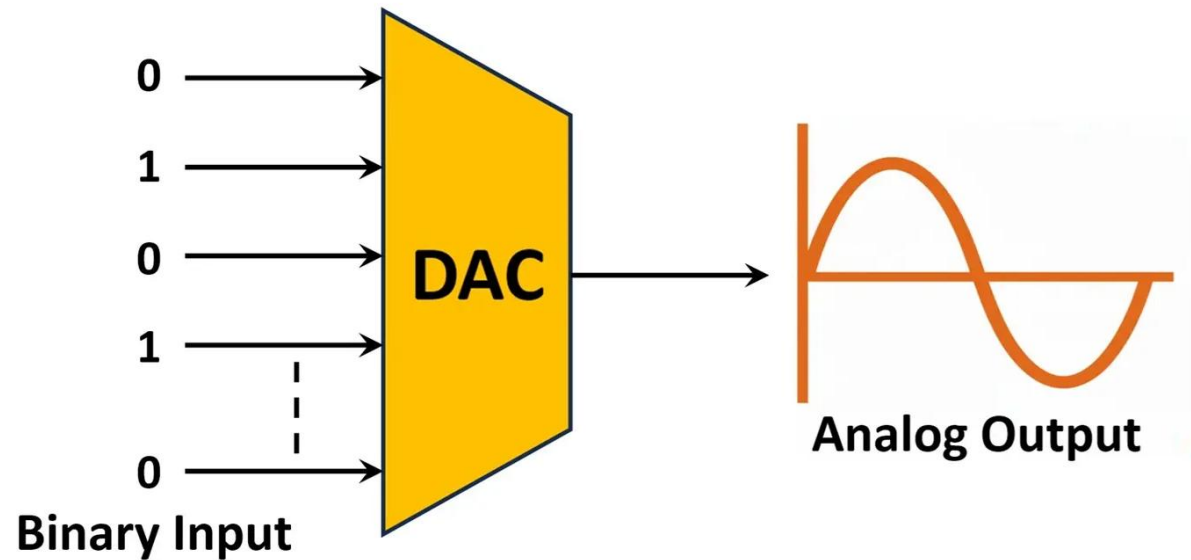
Existen dos formas muy utilizadas de lograr convertir valores digitales en señales analógicas en sistemas embebidos:

- Mediante un DAC.
- Mediante PWM.

Ambos métodos permiten que un sistema digital controle variables analógicas, pero lo hacen de maneras diferentes, con distintas ventajas y limitaciones.

DAC

Un DAC (*Digital to Analog Converter*) es un circuito que convierte un número digital en un voltaje analógico.



Funcionamiento del DAC

Un DAC típicamente funciona usando pesos binarios:

$$V_{out} = b_2 \frac{V_{ref}}{2} + b_1 \frac{V_{ref}}{4} + b_0 \frac{V_{ref}}{8}$$

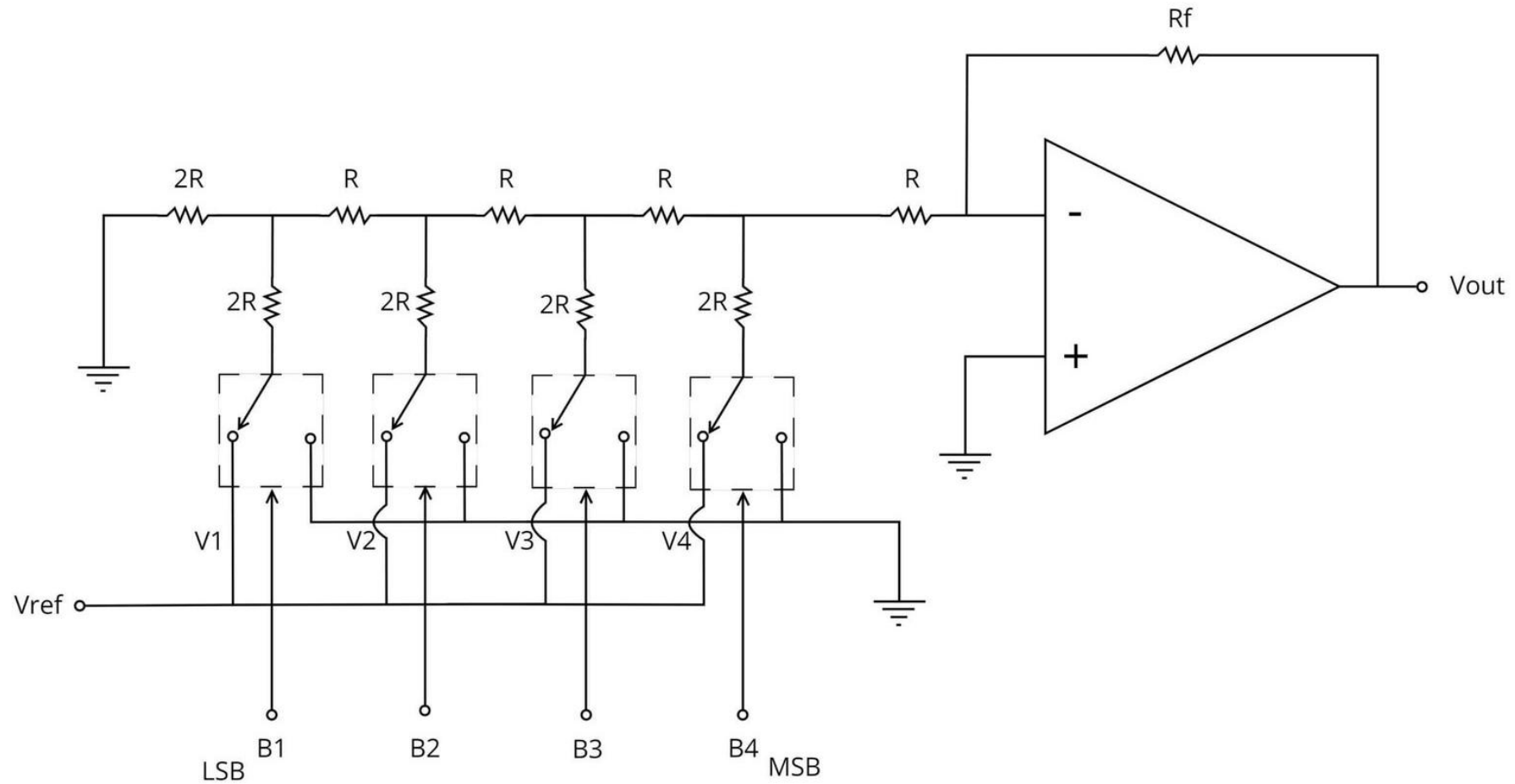
donde los b_i son los bits.

Funcionamiento del DAC

Para un DAC de n bits, la salida máxima (cuando todos los bits son 1) siempre sigue esta regla:

$$V_{o_{m\acute{a}x}} = V_{ref} \frac{2^n - 1}{2^n}$$

El valor máximo de salida siempre es un escalón (LSB) menos que el voltaje de referencia



Ejemplo para un
DAC de **3 bits**.

Digital	Voltaje
000	0
001	0.4125 V
010	0.825 V
011	1.2375 V
100	1.65 V
101	2.0625 V
110	2.4750 V
111	2.8875 V

Características del DAC

Resolución

La resolución indica cuántos niveles analógicos diferentes puede generar el DAC. Se expresa en bits (N).

$$Niveles = 2^N$$

Esto significa que el rango de voltaje de salida se divide en muchos niveles discretos. La resolución determina **qué tan pequeño** es el cambio de voltaje que el DAC puede producir en su salida.

Características del ADC

Tamaño del paso (LSB)

Representa el cambio mínimo de voltaje que puede generar el DAC.

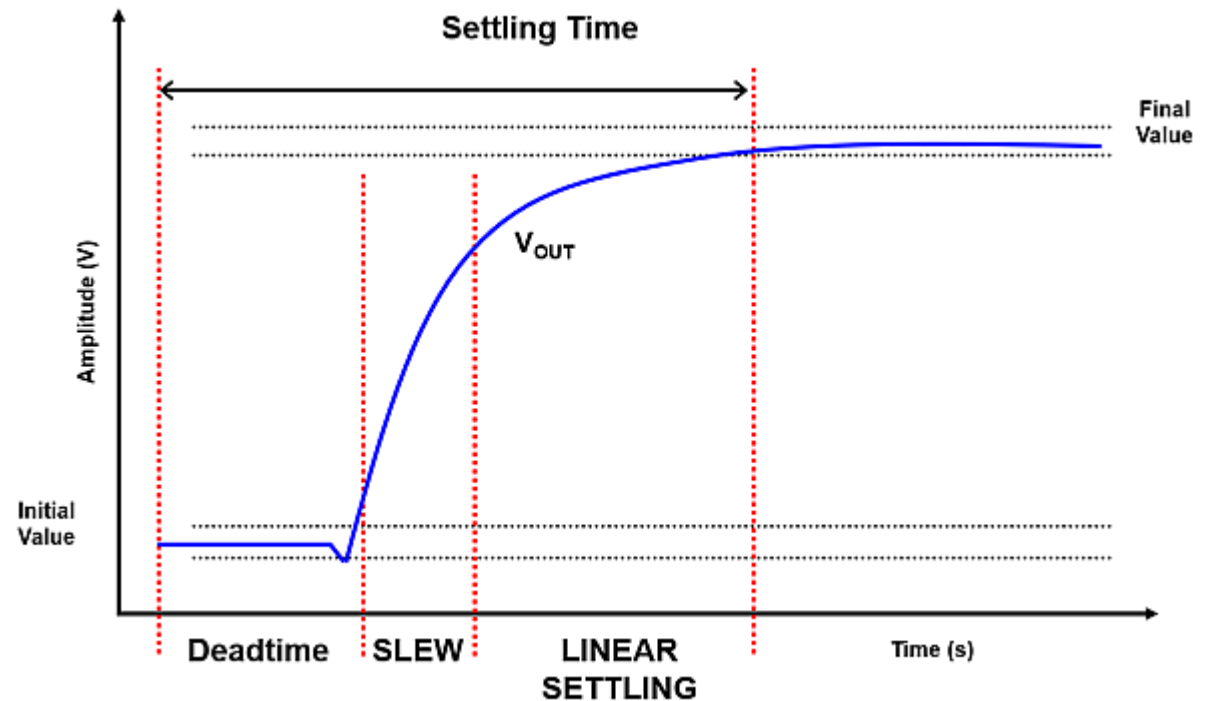
$$LSB = \frac{V_{ref}}{2^N}$$

Donde N es la resolución del DAC.

Características del DAC

Tiempo de establecimiento (Settling Time)

es el tiempo que tarda el DAC en estabilizar el voltaje después de cambiar el valor digital. En aplicaciones de alta velocidad, este parámetro es muy importante.



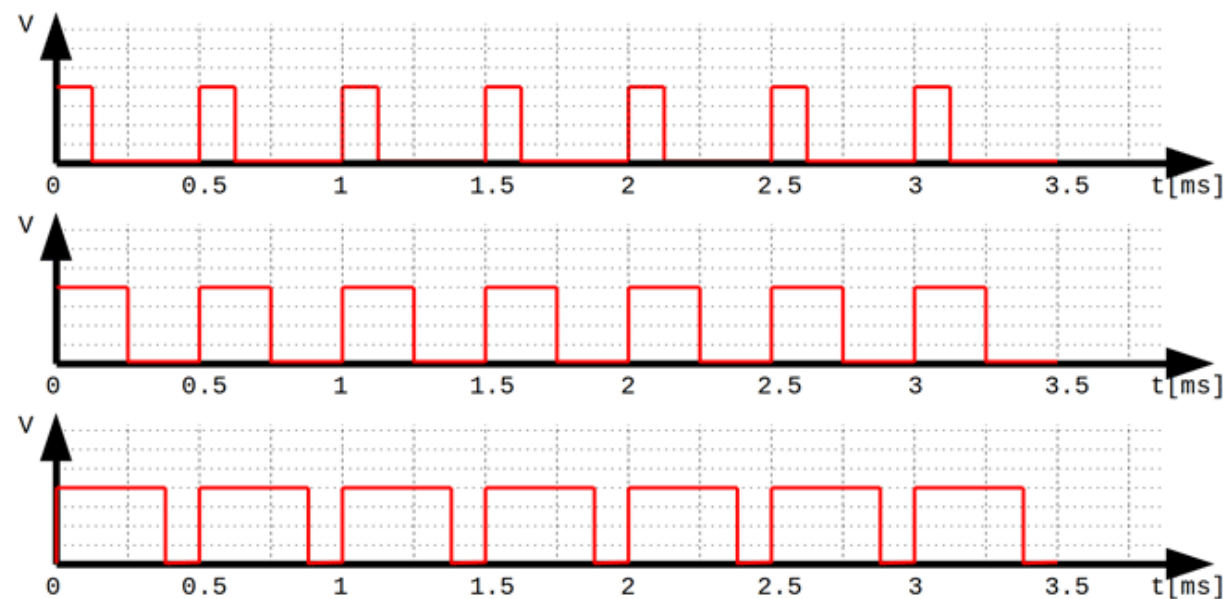
PWM

La PWM (*Pulse Width Modulation*) es una técnica en la que se varía el tiempo que una señal eléctrica permanece activa (nivel alto) frente al tiempo que permanece inactiva (nivel bajo) dentro de un ciclo constante. O sea, **la amplitud no cambia**, lo que cambia es el **ancho del pulso**.

Señal PWM

Una señal PWM es una onda cuadrada periódica caracterizada por:

- Periodo fijo.
- Amplitud fija.
- Ancho de pulso variable.



Parámetros fundamentales del PWM

Periodo (T):

Es el tiempo que tarda un ciclo completo de la señal.

$$T = t_{ON} + t_{OFF}$$

Frecuencia (f):

Es el número de ciclos por segundo.

$$f = \frac{1}{T}$$

Frecuencias comunes en sistemas embebidos

Aplicación	Frecuencia típica
LED dimming	500 Hz – 2 kHz
Motores DC	5 kHz – 20 kHz
Audio PWM	> 20 kHz
Servos	50 Hz

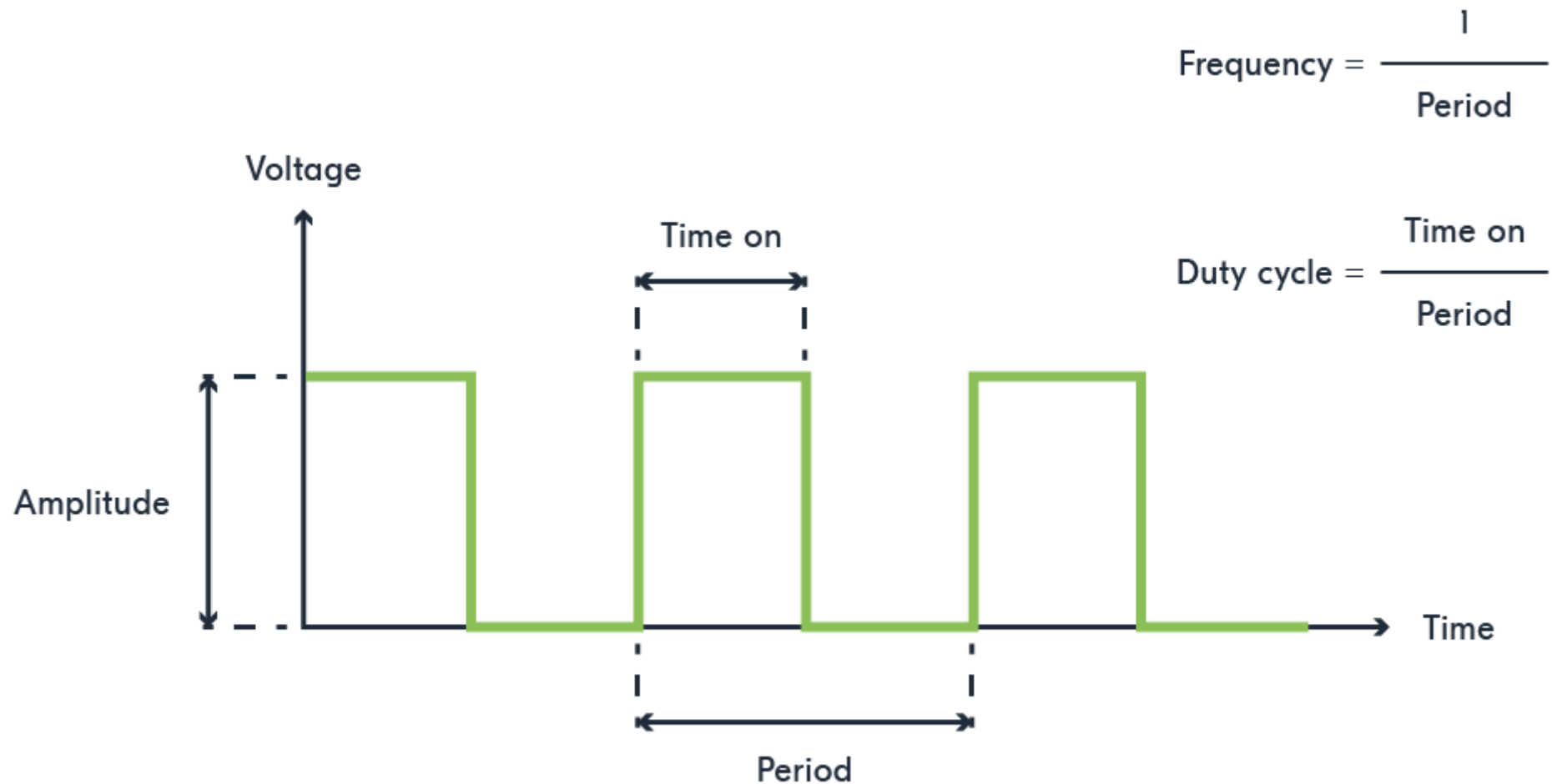
Parámetros fundamentales del PWM

Duty cycle (D):

Es la fracción del periodo en la que la señal está en **alto**.

$$D = \frac{t_{ON}}{T}$$

Normalmente se expresa en **porcentaje**: $D = \frac{t_{ON}}{T} \cdot 100$



50% duty cycle



75% duty cycle



25% duty cycle



Voltaje promedio del PWM

Cuando una señal PWM pasa por un **filtro** o por un sistema físico con **inercia**, el resultado es un **voltaje promedio**.

Ese voltaje es proporcional al duty cycle.

El voltaje promedio es:

$$V_{avg} = \frac{1}{T} \int_0^T V(t) dt$$

Voltaje promedio del PWM

Hay sistemas que no responden instantáneamente a cambios rápidos, sino que integran o promedian la energía que reciben.

En PWM estamos aplicando energía en pulsos muy rápidos, y muchos sistemas físicos no pueden seguir esos cambios, por lo que reaccionan al valor promedio.

La retina y el cerebro, por ejemplo, no responden instantáneamente a cambios rápidos de luz, el ojo humano integra la luz aproximadamente en ventanas de 10 – 20 ms (frecuencias entre 50 Hz y 100 Hz) y lo que percibe es el **promedio** de la luz emitida.

Voltaje promedio del PWM

Los motores tienen dos tipos de inercia:

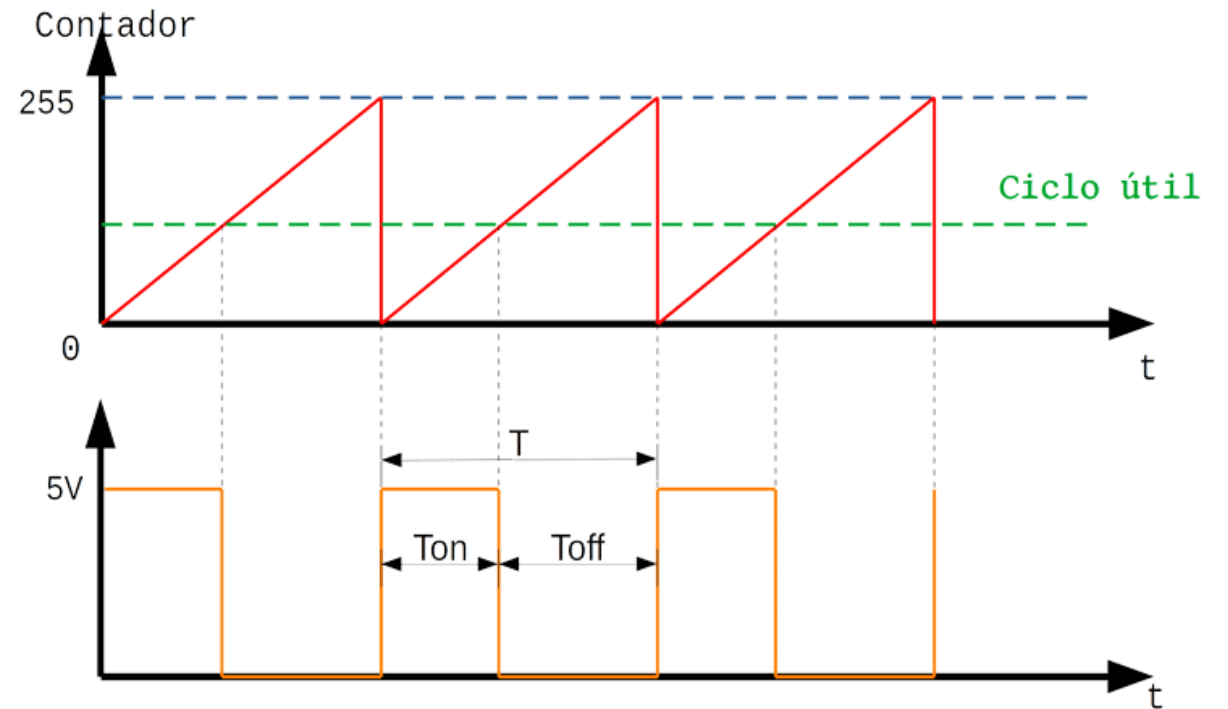
- Inercia eléctrica.
- Inercia mecánica.

Ambas hacen que el motor no responda instantáneamente a los pulsos PWM, por lo que responde de igual manera al voltaje promedio de la señal.

Cómo se genera el PWM internamente

Un PWM en hardware normalmente se genera con un timer y un comparador.

El comparador compara el valor del timer con un valor de referencia.



Resolución del PWM

La resolución depende de cuántos valores puede tomar el contador.

La resolución de PWM no se refiere a amplitud analógica directa, sino a cuántos **niveles de duty cycle** se pueden programar.

Más resolución significa un control más fino del duty cycle.

$$Niveles = 2^N$$

Ejemplo: $8 \text{ bits} \rightarrow 2^8 = 256$

DAC en el ESP32

El ESP32 tiene dos DACs integrados de 8 bits.

El voltaje máximo es de 3.3V y pueden usarse de forma independiente.

Se encuentran en:

- **DAC 1:** GPIO25
- **DAC 2:** GPIO26

El DAC del ESP32 está implementado como un DAC resistivo (R-2R ladder).

Modos de uso del DAC

En el ESP32 hay **tres** formas principales de usar el DAC.

DAC One-shot (valor fijo):

Este modo es el equivalente a poner un voltaje y dejarlo ahí.

Se usa cuando se quiere simplemente generar un voltaje fijo y cambiarlo ocasionalmente.

DAC Continuous (stream de datos):

Aquí el DAC recibe un buffer de muestras y las reproduce a una tasa fija.

Modos de uso del DAC

DAC Cosine generator

El ESP32 tiene un generador hardware de coseno.

Tiene menos flexibilidad, solo coseno y además tiene un control limitado.

Se usa principalmente para calibraciones, pruebas y señales internas.

PWM en el ESP32

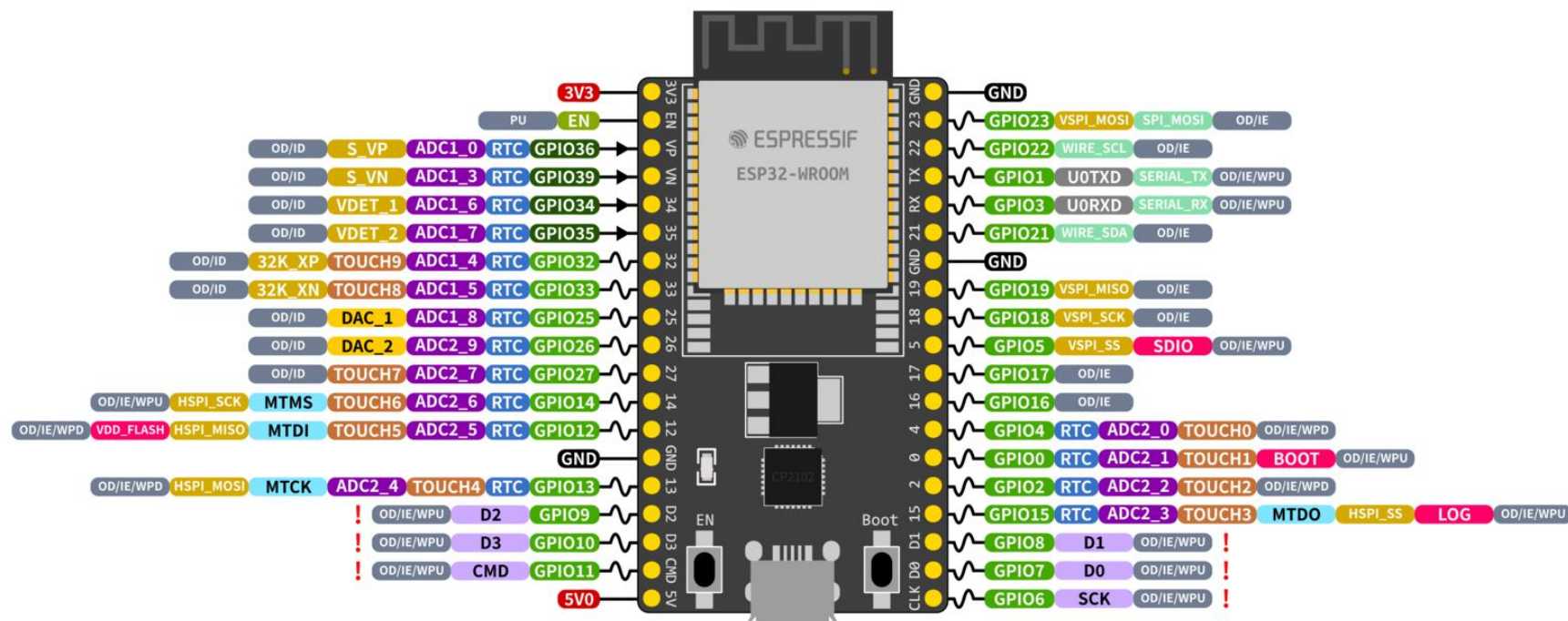
En el ESP32 se llama LEDC.

El ESP32 tiene 16 canales PWM y permite hasta 20 bits de resolución.

El duty cycle se expresa como un número entero, por ejemplo, para una resolución de 10 bits:

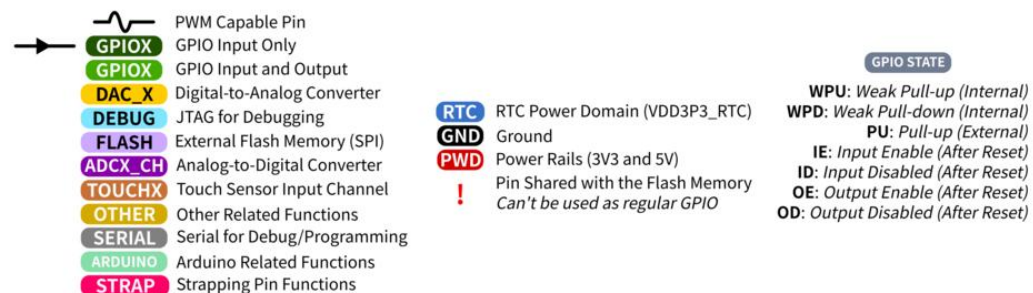
$$D = 100\% \rightarrow D = 1023$$

$$D = 50\% \rightarrow D = 512$$



ESP32 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth 4.2 BR/EDR and BLE
 520 KB SRAM (16 KB for cache)
 448 KB ROM
 34 GPIOs, 4x SPI, 3x UART, 2x I2C,
 2x I2S, RMT, LED PWM, 1 host SD/eMMC/SDIO,
 1 slave SDIO/SPI, TWAI®, 12-bit ADC, Ethernet



Configuración del DAC en modo continuo

```
#include <math.h>
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "driver/dac_continuous.h"
```

Configuración del DAC en modo continuo

```
dac_continuous_handle_t dac_handle;
dac_continuous_config_t config = {
    .chan_mask = DAC_CHANNEL_MASK_CH0,
    .desc_num = 4,
    .buf_size = 256,
    .freq_hz = 20000,
    .offset = 0,
    .clk_src = DAC_DIGI_CLK_SRC_DEFAULT,
    .chan_mode = DAC_CHANNEL_MODE_ALTER,
};

dac_continuous_new_channels(&config, &dac_handle);
dac_continuous_enable(dac_handle);
dac_continuous_write_cyclically(dac_handle, data_table, sizeof(data_table), NULL);
```

Configuración del DAC one shot mode



```
#include <stdint.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "driver/dac_oneshot.h"
```

Configuración del DAC one shot mode

```
dac_one-shot_handle_t dac_handle;  
dac_one-shot_config_t config = {  
    .chan_id = DAC_CHAN_0,  
};  
  
dac_one-shot_new_channel(&config, &dac_handle);
```

Escribir un valor en el DAC (one shot)



```
dac_oneshot_output_voltage(dac_handle, VALUE);
```

Configuración del DAC modo coseno



```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
  
#include "driver/dac_cosine.h"
```

Configuración del DAC modo coseno

```
dac_cosine_handle_t dac_handle;  
dac_cosine_config_t config = {  
  
    .chan_id = DAC_CHAN_0,  
    .freq_hz = 1000,  
    .clk_src = DAC_COSINE_CLK_SRC_DEFAULT,  
    .offset = 0,  
    .phase = DAC_COSINE_PHASE_0,  
    .atten = DAC_COSINE_ATTEN_DEFAULT,  
    .flags.force_set_freq = false,  
};  
  
dac_cosine_new_channel(&config, &dac_handle);  
dac_cosine_start(dac_handle);
```

Configuración del PWM



```
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"  
  
#include "driver/ledc.h"
```

Configuración del PWM

```
ledc_timer_config_t ledc_timer = {
    .speed_mode      = LEDC_LOW_SPEED_MODE,
    .timer_num       = LEDC_TIMER_0,
    .duty_resolution = LEDC_TIMER_12_BIT,
    .freq_hz         = 5000,
    .clk_cfg         = LEDC_AUTO_CLK,
};
ledc_timer_config(&ledc_timer);

ledc_channel_config_t ledc_channel = {

    .speed_mode = LEDC_LOW_SPEED_MODE,
    .channel     = LEDC_CHANNEL_0,
    .timer_sel   = LEDC_TIMER_0,
    .intr_type   = LEDC_INTR_DISABLE,
    .gpio_num    = 18,
    .duty        = 0, // Duty inicial
    .hpoint      = 0, // Punto de inicio del pulso: 0 (normal)
};
ledc_channel_config(&ledc_channel);
```

Configuración del duty cycle de PWM



```
ledc_set_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0, duty_cycle;  
ledc_update_duty(LEDCLOW_SPEED_MODE, LEDC_CHANNEL_0);
```